Aula 08

Android, navegação e ciclo de vida

Como organizar telas, acompanhar o app e trabalhar com dados no Flutter

- Activity e Fragment
- Navigator e rotas
- Estado do app e dos widgets
- Stack, Map e map()

Diego Possamai • Material original: Profª Tassiana Kautzmann

Fonte: Flutter Docs

O que vamos construir

Um catálogo pequeno será o exemplo ao longo da aula.

Uma lista de produtos

O usuário toca em um produto e abre seus detalhes.

Uma navegação previsível

Voltar retorna à lista. Finalizar um fluxo pode limpar o histórico.

Um app que reage ao Android

Vamos observar o que acontece ao sair do app e retornar.

Três significados de estado

Cada pergunta pede uma ferramenta diferente.

Pergunta	Tipo de estado	Ferramenta
O app está em primeiro plano?	Ciclo de vida do aplicativo	AppLifecycleListener
Qual tela está aberta?	Navegação e histórico	Navigator / Router
Qual produto foi escolhido?	Dados da interface	State / setState()

Fonte: Flutter Docs

Como o Android executa um app

O sistema operacional coordena a execução e o acesso ao dispositivo.

Processo e isolamento

Em geral, cada app executa em seu próprio processo, com dados e permissões isolados.

Componentes e eventos

O Android ativa componentes quando o usuário ou outro app solicita uma ação.

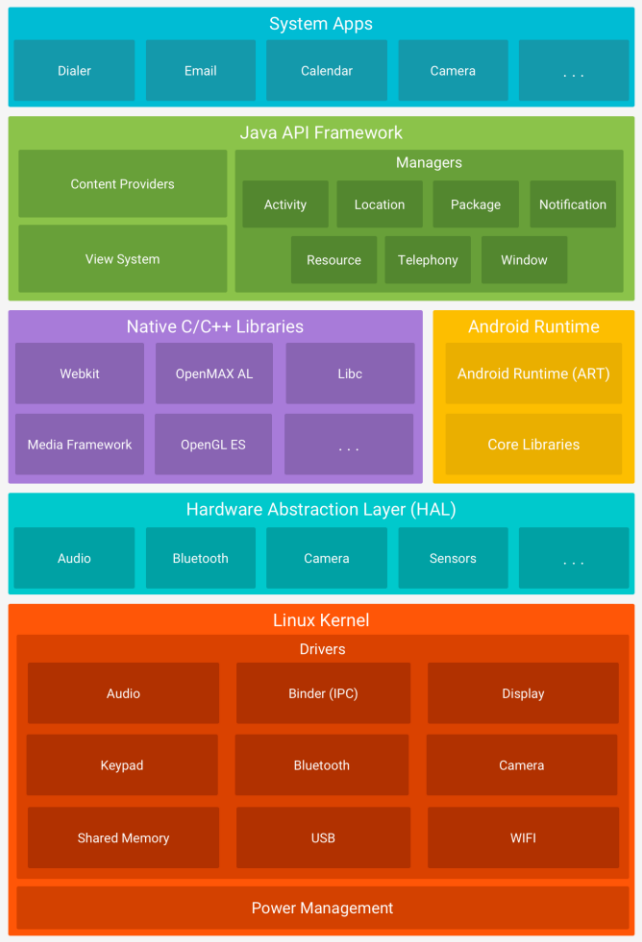
Memória e segundo plano

O processo pode terminar para liberar memória. Dados apenas em RAM podem desaparecer.

Fonte: Android Developers

Arquitetura do Android

Diagrama oficial. A leitura começa na base e sobe até os aplicativos.



Fonte: Android Developers

Leitura das camadas

O aplicativo usa serviços da plataforma para chegar ao hardware.

Kernel Linux e HAL

O kernel gerencia recursos. A HAL padroniza interfaces para componentes de hardware.

Runtime, bibliotecas e framework

ART executa código DEX. APIs oferecem janelas, notificações, câmera e outros serviços.

Onde o Flutter entra

O app inclui uma integração Android e o engine Flutter. Dart não vira uma coleção de Views Android.

Fonte: Android Developers

Componentes de um app Android

Visão geral para entender o papel da Activity.

Componente	Responsabilidade	Exemplo
Activity	Hospedar interação visual	Janela do catálogo
Service	Executar trabalho sem uma tela própria	Reprodução de áudio
BroadcastReceiver	Receber eventos compatíveis com as regras do sistema	Reagir a um evento
ContentProvider	Disponibilizar dados por uma interface padronizada	Acesso a contatos

Service não cria uma thread automaticamente. Trabalho em segundo plano tem restrições.

Fonte: Android Developers

Activity: a entrada visual do app

Uma Activity hospeda a interface e participa do ciclo de vida do Android.

Uma Activity pode conter várias telas

Um app moderno pode usar uma única Activity e navegar dentro dela.

O Android controla sua existência

A Activity pode iniciar, perder foco, deixar de aparecer ou ser recriada.

No nosso projeto Flutter

MainActivity normalmente herda de FlutterActivity e hospeda o conteúdo Flutter.

Fonte: Android Developers

Fragment: uma parte da interface nativa

Conceito do Android com AndroidX, útil para compreender apps nativos e híbridos.

Um componente hospedado

O Fragment precisa de uma Activity ou de outro Fragment como hospedeiro.

Interface e ciclo de vida próprios

Pode representar a lista ou o painel de detalhes em um app Android nativo.

Na interface Flutter

Telas e partes reutilizáveis são widgets. Um widget Flutter não é um Fragment Android.

Fonte: Android Developers

Android nativo e Flutter

São papéis parecidos, sem equivalência direta entre as classes.

Necessidade	Android nativo	Flutter
Hospedar o conteúdo	Activity	FlutterActivity no Android
Compor uma interface	Views, Fragments ou Compose	Widgets e composição
Navegar internamente	Navigation / FragmentManager	Navigator / Router
Acessar recurso nativo	APIs Android	Plugin ou platform channel

FlutterFragment existe para integração com um app Android existente. Não é necessário para cada tela Flutter.

Fonte: Flutter Docs

Onde fica a MainActivity

Arquivo Kotlin da pasta android/ do projeto. O pacote varia conforme o app.

```
package com.example.catalogo

import io.flutter.embedding.android.FlutterActivity

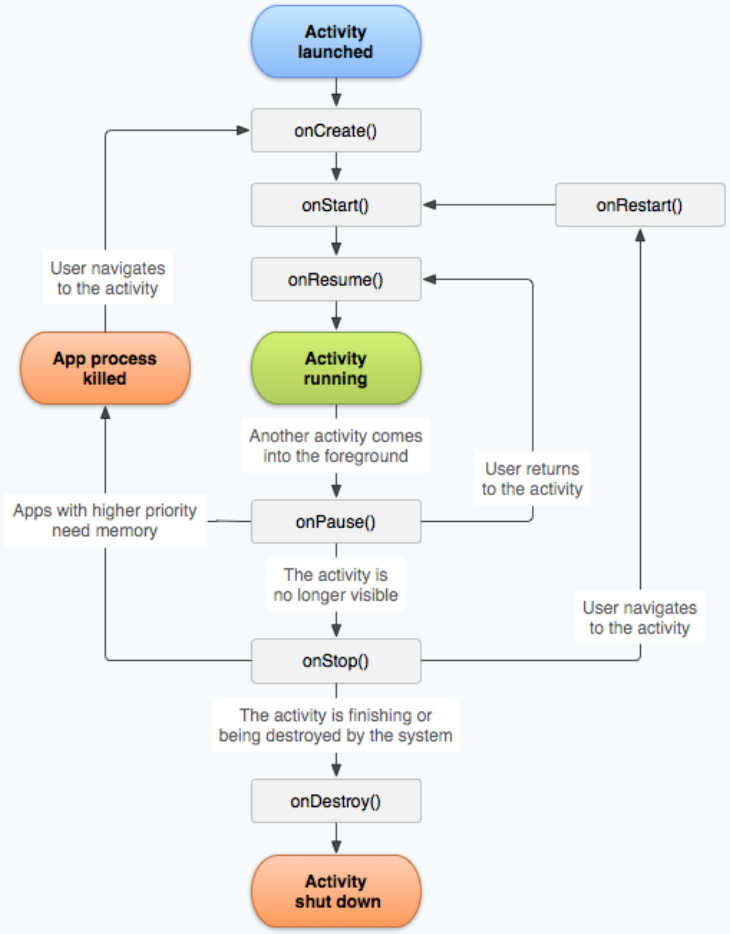
class MainActivity : FlutterActivity()
```

As telas ListaPage e DetalhesPage serão classes Dart em lib/. Não criamos uma Activity para cada uma.

Fonte: Flutter Docs

Ciclo de vida da Activity

Diagrama oficial. Os métodos são callbacks que o sistema chama.



Fonte: Android Developers

Callbacks: entrada e retorno

O sistema chama estes métodos. O aplicativo implementa as reações.

Callback	Quando ocorre	Uso típico
onCreate()	Criação da instância	Inicializar a interface
onStart()	A Activity se torna visível	Preparar recursos ligados à visibilidade
onResume()	Entra no estado Resumed	Retomar interação
onRestart()	Retorno de uma Activity parada	Passa depois por onStart()

Fonte: Android Developers

Callbacks: saída e encerramento

Visibilidade e execução podem mudar sem o usuário fechar o aplicativo.

Callback	O que significa	Cuidado
onPause()	Sai do estado Resumed	Ainda pode estar visível
onStop()	Deixa de estar visível	Libere recursos que dependem de visibilidade
onDestroy()	A instância será destruída	Pode ocorrer por recriação, não só por saída

Se o processo morrer abruptamente, onDestroy() pode não acontecer. Não deixe toda a gravação para o final.

Fonte: Android Developers

Um exemplo com o botão Início

Sequência típica, supondo que a Activity permaneça na memória.

Abrir o app

onCreate(), depois onStart() e onResume().

Pressionar Início

onPause() e depois onStop(), quando a Activity fica invisível.

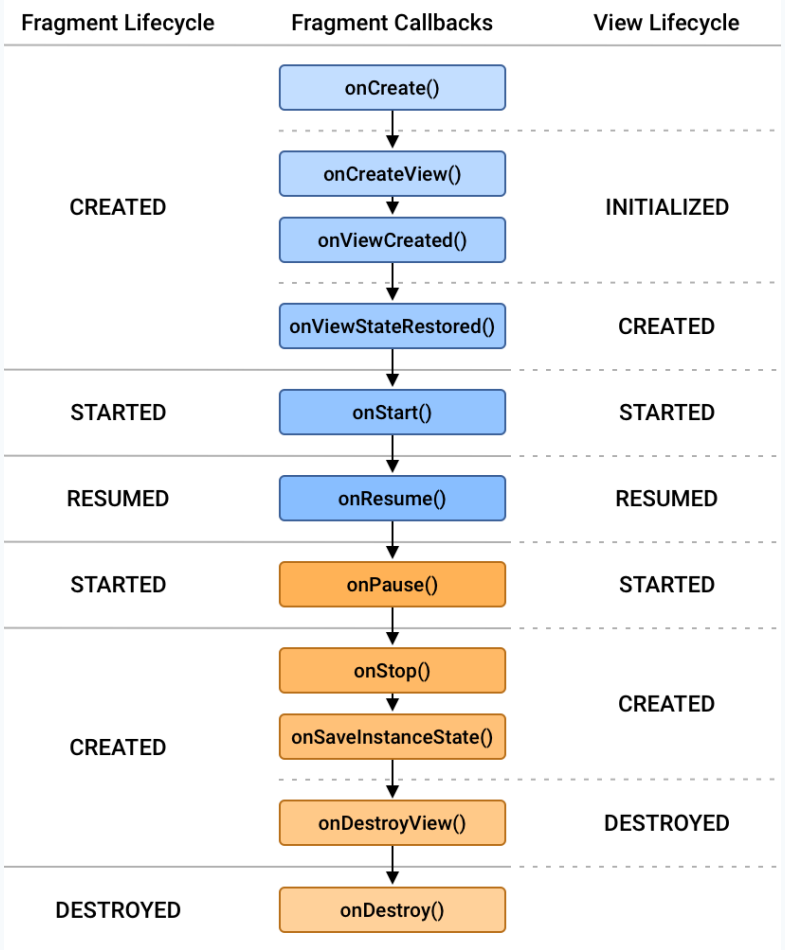
Retornar pelo seletor de apps

onRestart(), onStart() e onResume(). Se o processo morreu, haverá nova criação.

Fonte: Android Developers

Fragment e sua View

Diagrama oficial. O Fragment e sua interface têm ciclos de vida relacionados.



Fonte: Android Developers

Por que a View do Fragment importa

O objeto Fragment pode continuar existindo após perder sua View.

Criar e preparar a interface

onCreateView() cria a View. onViewCreated() permite configurar seus elementos.

Acompanhar presença na tela

onStart(), onResume(), onPause() e onStop() acompanham as transições.

Liberar referências

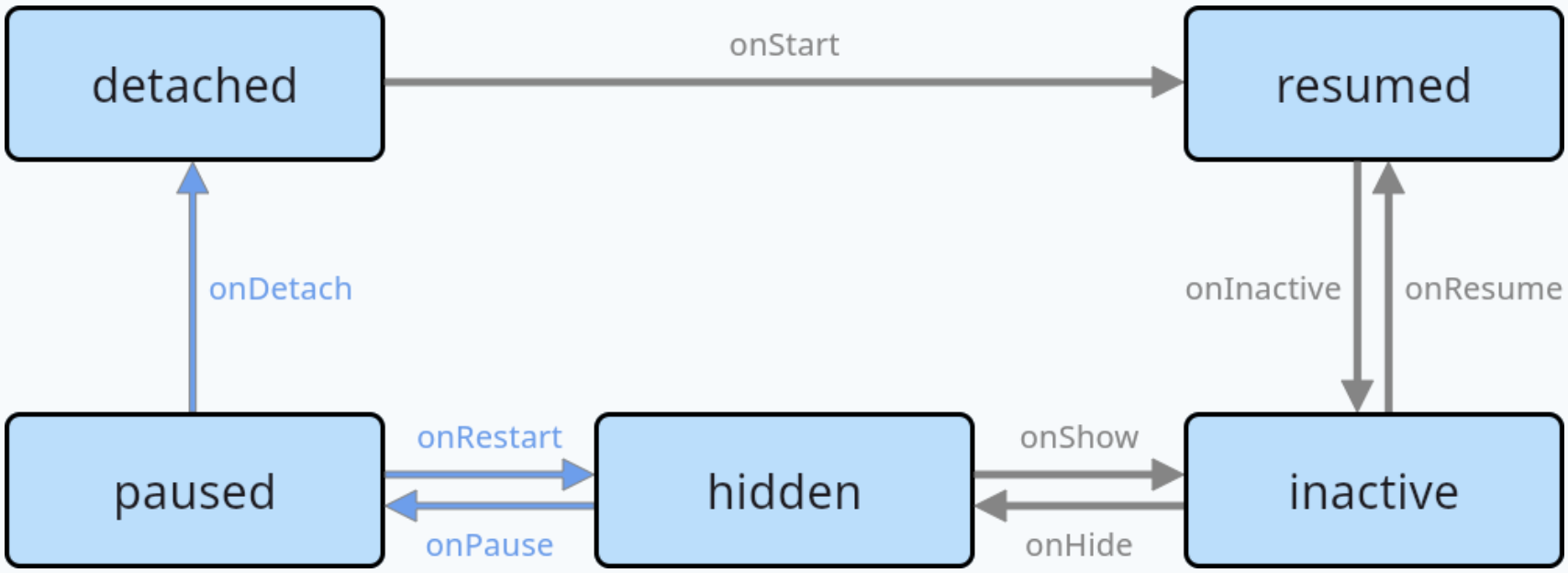
onDestroyView() encerra a View. onDestroy() encerra a instância do Fragment.

No Flutter, use widgets para composição e State para recursos vinculados à árvore.

Fonte: Android Developers

Ciclo de vida do app Flutter

Diagrama oficial. As transições azuis são específicas de Android e iOS.



Fonte: Flutter Docs

AppLifecycleState

Os nomes do Flutter não correspondem um a um aos callbacks Android.

Estado	Significado simplificado
resumed	App visível e com foco de entrada.
inactive	Há uma View visível, mas sem foco.
hidden	Todas as Views estão ocultas.
paused	Sem visibilidade nem entrada, em Android/iOS.
detached	Engine sem View hospedeira associada.

Fonte: Flutter Docs

Correspondência no Android

O foco da janela também participa das transições do Flutter.

Android	Flutter	Observação
onResume() + foco	resumed	onResume() sozinho não garante foco
onPause() ou perda de foco	inactive	Pode ocorrer com o app ainda visível
onStop()	paused	hidden aparece na transição móvel
Encerramento abrupto	Sem evento garantido	Algumas notificações podem faltar

Fonte: Flutter Docs

Como ouvir o ciclo de vida

AppLifecycleListener simplifica o acompanhamento no código Dart.

Criar uma assinatura

Em initState(), crie o listener e informe o callback onStateChange.

Reagir à mudança

Compare o estado recebido e execute uma ação adequada ao recurso.

Encerrar a assinatura

Em dispose(), chame dispose() no listener. Isso evita uma assinatura esquecida.

O exemplo seguinte mostra o estado na tela e o registra no console.

Fonte: Flutter Docs

Exemplo de ciclo de vida: início

Em lib/main.dart. As três partes formam um único exemplo executável.

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(
  home: CicloPage(),
));

class CicloPage extends StatefulWidget {
  const CicloPage({super.key});
  @override
  State<CicloPage> createState() => _CicloPageState();
}
```

CicloPage cria um State para manter a assinatura enquanto a página existir.

Fonte: Flutter Docs

Exemplo de ciclo de vida: assinatura

Continuação do mesmo arquivo, após a classe CicloPage.

```
class _CicloPageState extends State<CicloPage> {
  late final AppLifecycleListener listener;
  String estado = 'Aguardando evento';

  @override
  void initState() {
    super.initState();
    listener = AppLifecycleListener(
      onStateChange: (novo) {
        debugPrint('App: ${novo.name}');
        setState(() => estado = novo.name);
      },
    );
  }
```

O callback recebe AppLifecycleState. A classe continua no próximo slide.

Fonte: Flutter Docs

Exemplo de ciclo de vida: interface

Continuação de _CicloPageState. Este trecho fecha a classe.

```
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Ciclo do app')),
    body: Center(child: Text(estado)),
  );

  @override
  void dispose() {
    listener.dispose();
    super.dispose();
  }
}
```

Execute no Android, pressione Início e retorne. Observe a tela e o console.

Fonte: Flutter Docs

Reações úteis ao estado do app

A ação depende do recurso que o aplicativo utiliza.

Ao perder foco

Em inactive, uma interface de câmera pode precisar suspender a captura. Consulte o plugin.

Ao voltar à interação

Em resumed, retome o recurso ou atualize dados que podem ter ficado antigos.

Enquanto estiver oculto

Não presuma que timers e tarefas Dart continuarão funcionando indefinidamente.

Salvar alterações importantes deve fazer parte do fluxo de dados do app.

Fonte: Flutter Docs

Alternativa: WidgetsBindingObserver

Trechos dentro de um State com o mixin WidgetsBindingObserver.

```
@override
void initState() {
  super.initState();
  WidgetsBinding.instance.addObserver(this);
}
@override
void didChangeAppLifecycleState(AppLifecycleState s) {
  debugPrint('App: ${s.name}');
}
@override
void dispose() {
  WidgetsBinding.instance.removeObserver(this);
  super.dispose();
}
```

Declaração: class _PaginaState extends State<Pagina> with WidgetsBindingObserver.
Escolha uma das duas abordagens.

Fonte: Flutter Docs

Ciclo de vida de um State

Esses métodos acompanham um objeto da árvore Flutter.

Método	Responsabilidade
initState()	Inicialização única para esta instância de State.
didChangeDependencies()	Reagir a dependências herdadas.
build()	Descrever a interface. Pode executar muitas vezes.
didUpdateWidget()	Reagir à nova configuração enviada pelo pai.
dispose()	Liberar recursos ao remover definitivamente o State.

Fonte: Flutter Docs

Navegar, ocultar e reconstruir

São acontecimentos diferentes, mesmo quando a imagem na tela muda.

push() cobre a página anterior

Com MaterialPageRoute padrão, seu State normalmente continua montado.

Pressionar Início oculta o app

O app muda de ciclo de vida. Isso não significa executar dispose() em cada página.

setState() pede uma reconstrução

Atualiza a interface. Não reinicia a Activity nem muda o estado resumed/paused.

Para observar cobertura e retorno de uma rota, existem RouteObserver e RouteAware.

Fonte: Flutter Docs

Estado em memória e dados salvos

Escolha a estratégia conforme o tempo pelo qual o dado precisa existir.

Necessidade	Android nativo	Flutter
Recriação da interface	ViewModel / estado salvo	State e arquitetura apropriada
Restaurar estado de UI	onSaveInstanceState / SavedStateHandle	RestorationMixin e widgets restorable
Guardar dados duráveis	Banco, arquivo ou servidor	Banco, arquivo ou servidor

ViewModel e State não substituem persistência. Navegação comum não restaura automaticamente o app após morte do processo.

Fonte: Android Developers

Navigator e a pilha de rotas

Uma rota representa uma página ou outra apresentação gerenciada pelo Navigator.

push() adiciona uma rota

Lista fica abaixo. Detalhes entra no topo e passa a aparecer.

pop() remove a rota atual

Ao remover Detalhes, a Lista volta a aparecer.

A raiz é o início do fluxo

O comportamento de Voltar depende da pilha e da integração com a plataforma.

Fonte: Flutter Docs

Exemplo de histórico

Leia cada pilha da esquerda para a direita. O último item é o topo.

Ação	Pilha de rotas	Tela visível
Abrir o app	[Lista]	Lista
Abrir Café	[Lista, Detalhes]	Detalhes de Café
Abrir ajuda	[Lista, Detalhes, Ajuda]	Ajuda
Voltar	[Lista, Detalhes]	Detalhes de Café

Fonte: Flutter Docs

Miniapp: entrada e primeira página

Novo exemplo em lib/main.dart. As próximas quatro partes formam o app.

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(
  home: ListaPage(),
));

class ListaPage extends StatelessWidget {
  const ListaPage({super.key});
  @override
  Widget build(BuildContext context) {
    const produtos = ['Café', 'Pão', 'Leite'];
```

O BuildContext desta página está abaixo do MaterialApp e encontra seu Navigator.

Fonte: Flutter Docs

Miniapp: lista e push()

Continuação de ListaPage.build(). Este trecho fecha o método e a classe.

```
    return Scaffold(
      appBar: AppBar(title: const Text('Produtos')),
      body: ListView(children: produtos.map((nome) {
        return ListTile(
          title: Text(nome),
          onTap: () => Navigator.push<void>(context,
            MaterialPageRoute<void>(
              builder: (_) => DetalhesPage(nome: nome),
            ),
          ),
        );
      }).toList()),
    );
  }
}
```

Ao tocar em Café, o construtor de DetalhesPage recebe a String Café.

Fonte: Flutter Docs

Miniapp: dados no construtor

Após ListaPage, declare a página de detalhes.

```
class DetalhesPage extends StatelessWidget {
  const DetalhesPage({super.key, required this.nome});
  final String nome;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(nome)),
```

required exige o nome ao criar a página. A classe continua no próximo slide.

Fonte: Flutter Docs

Miniapp: pop() e retorno

Continuação do Scaffold da página de detalhes.

```
      body: Center(
        child: FilledButton(
          onPressed: () => Navigator.pop(context),
          child: const Text('Voltar para a lista'),
        ),
      ),
    );
  }
}
```

pop() remove Detalhes. O botão de voltar da AppBar também permite retornar.

Fonte: Flutter Docs

Uma página pode devolver um resultado

Trecho de callback async em uma página que abre uma seleção.

```
final escolhido = await Navigator.push<String>(
  context,
  MaterialPageRoute<String>(
    builder: (_) => const EscolhaPage(),
  ),
);
if (!context.mounted || escolhido == null) return;
ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(content: Text('Escolhido: $escolhido')),
);
```

EscolhaPage deve executar Navigator.pop(context, "Café"). Voltar sem escolher retorna null.

Fonte: Flutter Docs

Substituir a rota atual

Trecho de callback após concluir uma etapa de um fluxo.

```
Navigator.pushReplacement<void, void>(
  context,
  MaterialPageRoute<void>(
    builder: (_) => const HomePage(),
  ),
);
```

Exemplo: [Login] passa a [Home]. Apenas a rota atual é substituída. HomePage é a página de destino do seu app.

Fonte: Flutter Docs

Limpar as rotas anteriores

pushAndRemoveUntil() adiciona uma rota e testa as anteriores.

```
Navigator.pushAndRemoveUntil<void>(
  context,
  MaterialPageRoute<void>(
    builder: (_) => const HomePage(),
  ),
  (route) => false,
);
```

Com false para todas as rotas anteriores, sobra apenas Home. Limpar a pilha não autentica nem encerra uma sessão.

Fonte: Flutter Docs

Manter uma rota pelo nome

ModalRoute.withName() cria um predicado para testar o nome da rota.

```
Navigator.pushAndRemoveUntil<void>(
  context,
  MaterialPageRoute<void>(
    builder: (_) => const ResumoPage(),
  ),
  ModalRoute.withName('/'),
);
```

Se a pilha era [/, Cadastro, Confirmação], fica [/, Resumo]. A rota / permanece abaixo de Resumo. Se não existir, as anteriores serão removidas.

Fonte: Flutter Docs

Rotas nomeadas: quando aparecem

MaterialApp.routes continua disponível, mas tem limitações.

Em exemplos e projetos existentes

Uma String, como /detalhes, identifica um construtor de página.

Para esta aula

Navigator com MaterialPageRoute mantém os exemplos explícitos e simples.

Em navegação mais complexa

A documentação indica Router ou pacotes como go_router, especialmente para URLs e deep links.

Rotas nomeadas não são a recomendação geral para a maioria dos novos apps.

Fonte: Flutter Docs

Rotas nomeadas: leitura de código

Exemplo de configuração no MaterialApp e chamada dentro de uma página.

```
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (_) => const ListaPage(),
    '/ajuda': (_) => const AjudaPage(),
  },
)

// Dentro de um callback abaixo do MaterialApp:
Navigator.pushNamed(context, '/ajuda');
```

AjudaPage representa uma página definida no projeto. O mapa routes associa nomes a funções construtoras.

Fonte: Flutter Docs

Pilha de rotas e widget Stack

A palavra pilha aparece em assuntos diferentes.

Navigator	Stack
Organiza histórico de rotas.	Sobrepõe widgets dentro de um layout.
push() abre uma rota.	children define os elementos sobrepostos.
pop() retorna à rota anterior.	Positioned posiciona um filho.
Exemplo: Lista e Detalhes.	Exemplo: ícone com número de notificações.

Fonte: Flutter Docs

Stack: exemplo sem imagens externas

Trecho para o body de um Scaffold. O número fica sobre o ícone.

```
Center(
  child: SizedBox(
    width: 100, height: 100,
    child: Stack(children: [
      const Icon(Icons.shopping_cart, size: 80),
      Positioned(
        right: 0, top: 0,
        child: CircleAvatar(
          radius: 16, child: const Text('3'),
        ),
      ),
    ]),
  ),
)
```

O primeiro filho fica atrás. O SizedBox define a área usada pelo Stack.

Fonte: Flutter Docs

Map e map(): nomes parecidos

O M maiúsculo muda o conceito.

Map<K, V> é uma coleção

Guarda pares de chave e valor. Exemplo: produto["nome"].

Iterable.map() é uma transformação

Produz um resultado para cada elemento. Exemplo: nomes.map((n) => Text(n)).

No nosso catálogo

Um Map pode guardar dados. map() pode transformar uma lista em widgets.

Fonte: Dart API

Map: criar, ler e alterar

O tipo declara o que as chaves e os valores podem conter.

```
final produto = <String, String>{
  'nome': 'Café',
  'categoria': 'Bebida',
};

print(produto['nome']); // Café
produto['nome'] = 'Chá';
produto['origem'] = 'Brasil';
produto.remove('categoria');
print('Produto: ${produto['nome']}');
```

final impede trocar a referência produto. Os dados deste Map continuam mutáveis. Prefira literais tipados quando forem suficientes.

Fonte: Dart API

Map: inserção e atualização

putIfAbsent() e update() ajudam a expressar a intenção.

```
final estoque = <String, int>{'Café': 3};

estoque.putIfAbsent('Pão', () => 5);
estoque.putIfAbsent('Café', () => 99);
print(estoque['Café']); // 3

estoque.update('Café', (qtd) => qtd + 1);
estoque.update('Leite', (qtd) => qtd + 1,
  ifAbsent: () => 1);
print(estoque); // {Café: 4, Pão: 5, Leite: 1}
```

putIfAbsent preserva a chave existente. update sem ifAbsent exige que a chave exista.

Fonte: Dart API

Map: percorrer os dados

keys, values e entries oferecem diferentes formas de leitura.

```
final estoque = <String, int>{'Café': 3, 'Pão': 5};

for (final nome in estoque.keys) {
  print(nome);
}
for (final qtd in estoque.values) {
  print(qtd);
}
for (final item in estoque.entries) {
  print('${item.key}: ${item.value}');
}
estoque.forEach((nome, qtd) => print('$nome: $qtd'));
```

entries fornece pares MapEntry. forEach também é válido; a escolha depende da clareza do código.

Fonte: Dart API

Chave ausente e null safety

Mesmo um Map<String, int> pode retornar null ao procurar uma chave.

```
final estoque = <String, int>{'Café': 3};

int? encontrado = estoque['Leite']; // null
int quantidade = estoque['Leite'] ?? 0;

print(quantidade); // 0
print(estoque.containsKey('Leite')); // false
```

O operador [] retorna V? porque a chave pode não existir. ?? fornece um valor alternativo. Evite usar ! sem garantia.

Fonte: Dart API

Onde colocar o sinal de interrogação

A posição de ? determina o que pode ser nulo.

Tipo	O que pode ser null	Exemplo
Map<String, int>?	O próprio mapa	Map<String, int>? m = null;
Map<String, int?>	Um valor	{"Café": null}
Map<String?, int>	Uma chave	{null: 3}

Chaves nulas são permitidas pelo tipo, mas nomes de campos normalmente usam String não nula.

Fonte: Dart API

map(): transformar uma lista

A lista original permanece igual. toList() materializa os resultados.

```
final numeros = [1, 2, 3];

final dobros = numeros.map((n) => n * 2).toList();
final textos = numeros.map((n) => '$n').toList();

print(dobros);  // [2, 4, 6]
print(textos);  // strings: "1", "2", "3"
print(numeros); // [1, 2, 3]
```

map() retorna um Iterable de avaliação preguiçosa. A função executa quando os resultados são percorridos.

Fonte: Dart API

map(): dados viram widgets

Trecho dentro de build(). Os nomes geram os filhos da Column.

```
final nomes = ['Café', 'Pão', 'Leite'];

return Column(
  children: nomes.map((nome) {
    return Text(nome);
  }).toList(),
);
```

Para listas pequenas, a transformação é direta. Para listas longas, prefira ListView.builder.

Fonte: Dart API

map(): gerar botões

Cada callback usa o nome do item que originou o botão.

```
final nomes = ['Café', 'Pão', 'Leite'];

return Column(
  children: nomes.map((nome) => FilledButton(
    onPressed: () {
      debugPrint('Selecionado: $nome');
    },
    child: Text(nome),
  )).toList(),
);
```

O mesmo padrão aparece no catálogo: cada item sabe qual dado deve enviar para a próxima página.

Fonte: Dart API

Um modelo para dados de produto

Uma classe deixa nome e preço explícitos e tipados.

```
class Produto {
  const Produto(this.nome, this.preco);
  final String nome;
  final double preco;
}

const produtos = [
  Produto('Café', 8.50),
  Produto('Pão', 2.00),
];
```

Um modelo reduz erros como escrever uma chave diferente. Map continua útil para associações e dados dinâmicos.

Fonte: Dart API

Produtos em cards

Trecho para a interface usando a lista do slide anterior.

```
ListView(
  children: produtos.map((p) => Card(
    child: ListTile(
      title: Text(p.nome),
      subtitle: Text(
        'R\$ ${p.preco.toStringAsFixed(2)}',
      ),
    ),
  )).toList(),
)
```

toStringAsFixed(2) demonstra duas casas decimais. Formatação monetária localizada exige tratamento próprio.

Fonte: Dart API

Diagnóstico de erros comuns

O sintoma ajuda a localizar o conceito que precisa de ajuste.

Sintoma	O que verificar
Navigator não encontrado	O context está abaixo de MaterialApp/Navigator?
Tela anterior sumiu do histórico	Foi usado replacement ou remoção de rotas?
Dados somem após reiniciar	Os dados estavam apenas na memória?
Busca no Map retorna null	A chave existe? O nome foi escrito corretamente?
Estado do app não muda ao navegar	As rotas continuam dentro do mesmo app ativo.

Fonte: Flutter Docs

Revisão da aula

Cada ferramenta resolve uma parte do comportamento do aplicativo.

Android hospeda e coordena

Activity, componentes, processo e callbacks fazem parte da plataforma.

Flutter organiza interface e navegação

Widgets compõem a tela. Navigator administra rotas. State guarda dados locais.

Eventos e dados têm ferramentas próprias

AppLifecycleListener observa o app. Stack sobrepõe elementos. Map guarda pares e map() transforma coleções.

Referências: Android

Documentação oficial. Links completos também nas notas dos slides.

Arquitetura da plataforma
https://developer.android.com/guide/platform

Componentes e processo
https://developer.android.com/guide/components/fundamentals

Ciclo de vida da Activity
https://developer.android.com/guide/components/activities/activity-lifecycle

Fragment e ciclo de vida da View
https://developer.android.com/guide/fragments/lifecycle

Salvamento de estado
https://developer.android.com/topic/libraries/architecture/saving-states

Referências: Flutter

Documentação oficial. Links completos também nas notas dos slides.

Flutter para desenvolvedores Android
https://docs.flutter.dev/flutter-for/android-devs

Estados do aplicativo
https://api.flutter.dev/flutter/dart-ui/AppLifecycleState.html

AppLifecycleListener
https://api.flutter.dev/flutter/widgets/AppLifecycleListener-class.html

Ciclo de vida de State
https://api.flutter.dev/flutter/widgets/State-class.html

Navegação e rotas
https://docs.flutter.dev/ui/navigation

Referências: código e diagramas

Documentação oficial. Links completos também nas notas dos slides.

Stack
https://api.flutter.dev/flutter/widgets/Stack-class.html

Map<K, V>
https://api.dart.dev/dart-core/Map-class.html

Iterable.map()
https://api.dart.dev/dart-core/Iterable/map.html

Remoção de rotas
https://api.flutter.dev/flutter/widgets/Navigator/pushAndRemoveUntil.html

Predicado ModalRoute.withName()
https://api.flutter.dev/flutter/widgets/ModalRoute/withName.html

Diagramas: Android Developers (CC BY 4.0) e Flutter API. Créditos e URLs das imagens nas notas.